

## Department of Artificial Intelligence

|     |                                                                                                                                                                                        |   |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| III | <b>Aggregation</b><br>Aggregation Framework, Pipeline Operations- \$match, \$sort, \$group, Pipeline Operations-,\$project, \$unwind, \$limit, \$skip, MapReduce, Aggregation commands | 6 |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|

### CAE-1 QUERY for reference

Develop a query in MongoDB to insert multiple documents for a Customer collection. Further, develop another query to update discount of 10% on item purchased if item quantity is more than or equal to 10 items.

### Insert Multiple Documents into Customer Collection

```
db.Customer.insertMany([
```

```
{
  customerId: 1,
  name: "Alice",
  item: "Laptop",
  quantity: 5,
  price: 1200,
  discount: 0
},
{
  customerId: 2,
  name: "Bob",
  item: "Monitor",
  quantity: 12,
  price: 300,
  discount: 0
},
{
  customerId: 3,
```

### G H Raison College of Engineering & Management

(Formerly Known as G H Raison Institute of Engineering & Technology, Nagpur)

An Autonomous Institute Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur

Accredited by NAAC with "A+" Grade

Shraddha Park, B-37-39/1, MIDC, Hingna-Wadi Link Road, Nagpur-440016 (INDIA)

```
name: "Charlie",  
item: "Keyboard",  
quantity: 15,  
price: 50,  
discount: 0  
},  
{  
  customerId: 4,  
  name: "Diana",  
  item: "Mouse",  
  quantity: 8,  
  price: 25,  
  discount: 0  
}  
])
```

### Update Query: Apply 10% Discount if Quantity $\geq 10$

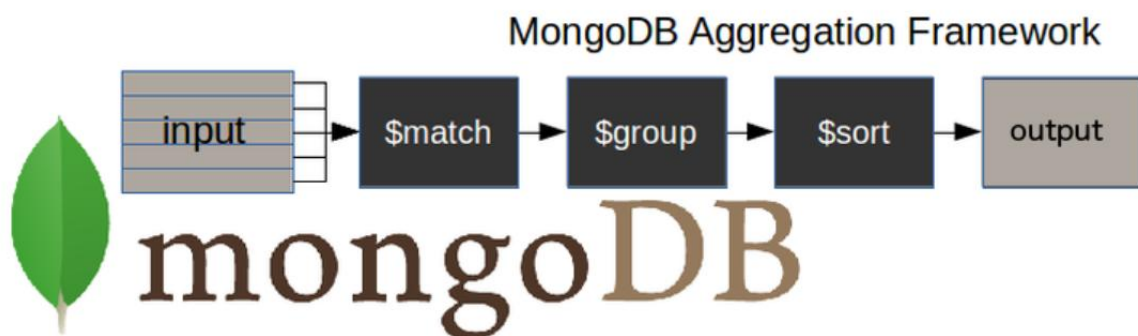
```
db.Customer.updateMany(  
  { quantity: { $gte: 10 } },  
  [  
    {  
      $set: {  
        discount: { $multiply: ["$price", 0.10] }  
      }  
    }  
  ]  
)
```

## UNIT : 3

## What is Aggregation in MongoDB?

Aggregation is a way of processing a large number of documents in a collection by means of passing them through different stages. The stages make up what is known as a pipeline. The stages in a pipeline can filter, sort, group, reshape and modify documents that pass through the pipeline.

## How does the MongoDB aggregation pipeline work?



**\$match** stage – filters those documents we need to work with, those that fit our needs

**\$group** stage – does the aggregation job

**\$sort** stage – sorts the resulting documents the way we require (ascending or descending)

The input of the pipeline can be a single collection, where others can be merged later down the pipeline.

The pipeline then performs successive transformations on the data until our goal is achieved.

Each stage of the pipeline transforms the documents as they pass through it and allows for operations like filtering, grouping, sorting, reshaping and performing calculations on the data.

### Single Purpose Aggregation

Single-purpose aggregation methods are designed for simple **analytical queries**. It is used when we need simple access to document like counting the number of documents or for finding all **distinct values** in a document

- **count()** – Returns the number of documents in a collection.
- **distinct()** – Retrieves unique values for a specified field.
- **estimatedDocumentCount()** – Provides an estimated count of documents.

## \$group

The \$group stage combines multiple documents with the same field, fields, or expression into a single document according to a group key. The result is one document per unique group key.

```
db.sales.insertMany([
  { "_id" : 1, "item" : "abc", "price" : Decimal128("10"), "quantity" : Int32("2"),
    "date" : ISODate("2014-03-01T08:00:00Z") },
  { "_id" : 2, "item" : "jkl", "price" : Decimal128("20"), "quantity" : Int32("1"),
    "date" : ISODate("2014-03-01T09:00:00Z") },
  { "_id" : 3, "item" : "xyz", "price" : Decimal128("5"), "quantity" : Int32("10"),
    "date" : ISODate("2014-03-15T09:00:00Z") },
  { "_id" : 4, "item" : "xyz", "price" : Decimal128("5"), "quantity" : Int32("20") ,
    "date" : ISODate("2014-04-04T11:21:39.736Z") },
  { "_id" : 5, "item" : "abc", "price" : Decimal128("10"), "quantity" : Int32("10") ,
    "date" : ISODate("2014-04-04T21:23:13.331Z") },
  { "_id" : 6, "item" : "def", "price" : Decimal128("7.5"), "quantity": Int32("5" ) ,
    "date" : ISODate("2015-06-04T05:08:13Z") },
  { "_id" : 7, "item" : "def", "price" : Decimal128("7.5"), "quantity": Int32("10") ,
    "date" : ISODate("2015-09-10T08:43:00Z") },
  { "_id" : 8, "item" : "abc", "price" : Decimal128("10"), "quantity" : Int32("5" ) ,
    "date" : ISODate("2016-02-06T20:20:13Z") },
])
```

The following aggregation operation uses the \$group stage to count the number of documents in the sales collection:

```
db.sales.aggregate( [
  {
    $group: {
      _id: null,
      count: { $count: { } }
    }
  }
])
```

The operation returns the following result:

```
{ "_id" : null, "count" : 8 }
```

## Retrieve Distinct Values

The following aggregation operation uses the `$group` stage to retrieve the distinct item values from the `sales` collection:

```
db.sales.aggregate( [ { $group : { _id : "$item" } } ] )
```

The operation returns the following result:

```
{ "_id" : "abc" }
```

```
{ "_id" : "jkl" }
```

```
{ "_id" : "def" }
```

```
{ "_id" : "xyz" }
```

## Group by Item Having

The following aggregation operation groups documents by the `item` field, calculating the total sale amount per item and returning only the items with total sale amount greater than or equal to 100:

```
db.sales.aggregate(  
[  
  // First Stage  
  {  
    $group :  
    {  
      _id : "$item",  
      totalSaleAmount: { $sum: { $multiply: [ "$price", "$quantity" ] } }  
    }  
  }  
]
```

```
}  
},  
// Second Stage  
{  
$match: { "totalSaleAmount": { $gte: 100 } }  
}  
]  
)
```

#### First Stage:

The `$group` stage groups the documents by `item` to retrieve the distinct item values. This stage returns the `totalSaleAmount` for each item.

#### Second Stage:

The `$match` stage filters the resulting documents to only return items with a `totalSaleAmount` greater than or equal to 100.

The operation returns the following result:

```
{ "_id" : "abc", "totalSaleAmount" : Decimal128("170") }  
{ "_id" : "xyz", "totalSaleAmount" : Decimal128("150") }  
{ "_id" : "def", "totalSaleAmount" : Decimal128("112.5") }
```

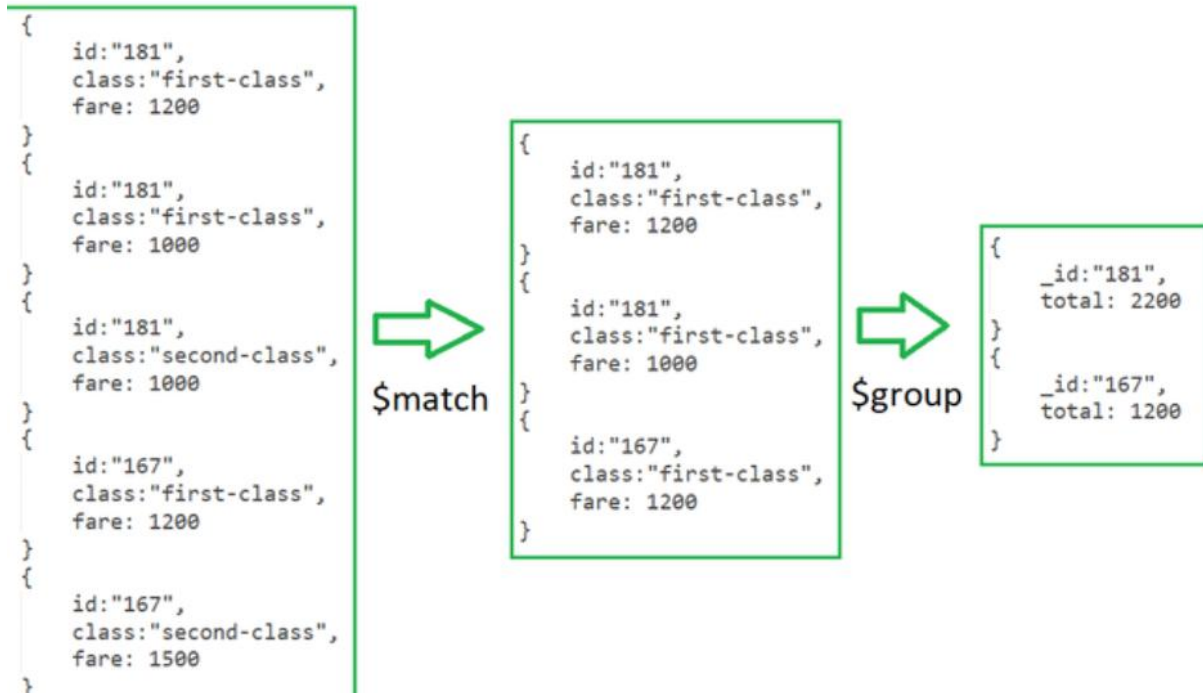
## 2. MongoDB Aggregation Pipeline

Mongodb Aggregation Pipeline consist of stages and each stage transforms the document. **It is a multi-stage pipeline and in each state the documents are taken as input to produce the resultant set of documents. In the next stage (ID available) the resultant documents are taken as input to produce output, this process continues till the last stage.**

The **basic pipeline stages** are defined below:

- filters that will operate like queries.
- the document transformation that modifies the resultant document.
- provide pipeline tools for **grouping and sorting documents**.
- Aggregation pipeline can also be used in sharded collection.

```
db.train.aggregate( [
  { $match: { class: "first-class" } },
  { $group: { _id: "id", total: { $sum: "$fare" } } } ] pipeline stages
)
```



### Explanation:

In the above example of a collection of "train fares". **\$match** stage filters the documents by the value in class field i.e. class: "first-class" in the first stage and passes the document to the second stage. In the Second Stage, the **\$group** stage groups the documents by the id field to calculate the sum of fare for each unique id.

```
db.train.aggregate([ { $group : { _id : "$id", total : { $sum : "$fare" } } } ])
```

Stage
Expression
Accumulator

## Aggregation Pipeline Method

To understand Aggregation Pipeline Method Let's imagine a collection named **users** with some documents for our examples.

### Query:

Query:

```
{
  "_id": ObjectId("60a3c7e96e06f64fb5ac0700"),
  "name": "Alice",
  "age": 30,
  "email": "alice@example.com",
  "city": "New York"
}
{
  "_id": ObjectId("60a3c7e96e06f64fb5ac0701"),
  "name": "Bob",
  "age": 35,
  "email": "bob@example.com",
  "city": "Los Angeles"
}
{
  "_id": ObjectId("60a3c7e96e06f64fb5ac0702"),
  "name": "Charlie",
  "age": 25,
  "email": "charlie@example.com",
  "city": "Chicago"
}
```

1. **\$group**: It Groups documents by the city field and calculates the average age using the **\$avg** accumulator.

```
db.users.aggregate([
  { $group: { _id: "$city", averageAge: { $avg: "$age" } } }
])
```

### Output:

[

```
[ { _id: 'New York', averageAge: 30 },  
  { _id: 'Chicago', averageAge: 25 },  
  { _id: 'Los Angeles', averageAge: 35 }  
]
```

2. **\$project**: Include or exclude fields from the output documents.

```
db.users.aggregate([  
  { $project: { name: 1, city: 1, _id: 0 } }  
])
```

**Output:**

```
[  
  { name: 'Alice', city: 'New York' },  
  { name: 'Bob', city: 'Los Angeles' },  
  { name: 'Charlie', city: 'Chicago' }  
]
```

3. **\$match**: Filter documents to pass only those that match the specified condition(s).

```
db.users.aggregate([  
  { $match: { age: { $gt: 30 } } }  
])
```

**Output:**

```
[  
  {  
    _id: ObjectId('60a3c7e96e06f64fb5ac0701'),  
    name: 'Bob',  
    age: 35,  
  }  
]
```

```
email: 'bob@example.com',  
city: 'Los Angeles'  
}  
]
```

3. \$sort: It Sorts documents based on field values

```
db.users.aggregate([  
  { $sort: { age: 1 } }  
])
```

**Output:**

```
[  
  {  
    _id: ObjectId('60a3c7e96e06f64fb5ac0702'),  
    name: 'Charlie',  
    age: 25,  
    email: 'charlie@example.com',  
    city: 'Chicago'  
  },  
  {  
    _id: ObjectId('60a3c7e96e06f64fb5ac0700'),  
    name: 'Alice',  
    age: 30,  
    email: 'alice@example.com',  
    city: 'New York'  
  },  
  {  
    _id: ObjectId('60a3c7e96e06f64fb5ac0701'),  
    name: 'Bob',  
    age: 35,  
    email: 'bob@example.com',  
    city: 'Los
```

```
Angeles'
```

```
}  
]
```

5. **\$limit**: Limit the number of documents passed to the next stage.

```
db.users.aggregate([  
  { $limit: 2 }  
])
```

**Output:**

```
[  
  {  
    _id: ObjectId('60a3c7e96e06f64fb5ac0700'),  
    name: 'Alice',  
    age: 30,  
    email: 'alice@example.com',  
    city: 'New York'  
  },  
  {  
    _id: ObjectId('60a3c7e96e06f64fb5ac0701'),  
    name: 'Bob',  
    age: 35,  
    email: 'bob@example.com',  
    city: 'Los Angeles'  
  }  
]
```

**Query:**

```
db.users.aggregate([  
  { $group: { _id: "$city", totalUsers: { $sum: 1 } } }  
])
```

**Output:**

**G H Raison College of Engineering & Management**

(Formerly Known as G H Raison Institute of Engineering & Technology, Nagpur)

An Autonomous Institute Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur

Accredited by NAAC with "A+" Grade

Shraddha Park, B-37-39/1, MIDC, Hingna-Wadi Link Road, Nagpur-440016 (INDIA)

```
[
  { _id: 'Los Angeles', totalUsers: 1 },
  { _id: 'New York', totalUsers: 1 },
  { _id: 'Chicago', totalUsers: 1 }
]
```

### How to use MongoDB Aggregation

To use MongoDB for aggregating data, follow below steps:

#### Step 1: Connect to MongoDB

Ensure you are connected to your MongoDB instance.

#### Step 2: Choose the Collection

Select the collection you want to perform aggregation on, such as **students**.

#### Step 3: Define the Aggregation Pipeline

Create an array of stages, like \$group to group documents and perform operations (e.g., calculate the average grade).

#### Step 4: Run the Aggregation Query

Use the aggregate method on the collection with your defined pipeline.

#### Example: Calculating Average Student Grade

This calculates the average grade of all students in the students collection.

#### Query:

```
db.students.aggregate([
  {
    $group: {
      _id: null,
      averageGrade: { $avg: "$grade" }
    }
  }
])
```

#### Output:

```
[
  { "_id": null, "averageGrade": 85 }
]
```

## Aggregation Pipeline Stages in MongoDB

The **aggregation** pipeline in **MongoDB** is a framework designed for data aggregation, inspired by data processing pipelines.

Documents enter a **multi-stage pipeline** that can transform them in various ways, including **filtering**, **grouping**, **reshaping**, and **modifying**.

**Each stage of the pipeline processes the documents as they pass through and performing an operation and passing the results to the next stage.**

There are different types of stages in the MongoDB called **aggregation pipeline stages**. The aggregation method takes the stages that work in the sequence. The data goes through multiple stages and returns the resulting output.

### Syntax:

```
db.<collection_name>.aggregate(pipeline, options)
```

or

```
db.<collection_name>.aggregate( [ { <stage1> },  
{<stage2>},... ] )
```

- A pipeline is a sequence of aggregation operations and stages. It is of array type. If the pipeline is not mentioned then it returns all the documents from the collection.
- Aggregation Stages always work in sequence means stage1 will execute first and pass the result to stage2 and so on.
- Options are optional and only used when you specify pipeline as an array.
- Aggregation is used to enhance performance by optimizing the query.
- Use variable and field names with a dollar sign(\$) and string without a dollar sign.

### Aggregation Stages

#### 1. \$addFields Stage

**\$addFields** stage is used to add new fields to the existing documents. It returns documents that contain all the existing fields and newly added fields.

Syntax :

```
db.<collection_name>.aggregate(
```

```

    {
      $addFields: { <newField1>: <expression>, <newField2>: <expression> ... }
    }
  });

```

#### Example:

Let's suppose we have to Add a new field `isActive` with the value `true` to every document in the `employee_details` collection using the MongoDB aggregation pipeline.

#### Query:

```

db.employee_details.aggregate([
  {
    $addFields: {"isActive": true }
  }
])

```

#### Output:

```

TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  powershell  + -  [ ]  [X]  [X]
> use company;
switched to db company
> db;
company
> db.employee_details.aggregate([
...   {
...     $addFields: {"isActive" : true }
...   }
... ])
{ "_id" : ObjectId("629101177017e180fa9ff11c"), "name" : { "firstName" : "Reema", "last
phone" : { "type" : "Home", "number" : "111-000-000" } }, "salary" : 85000, "doj" : ISO
skills" : [ "React", "MongoDB", "Javascript" ], "isActive" : true }
{ "_id" : ObjectId("6291185b7017e180fa9ff11d"), "name" : { "firstName" : "Seema", "last
phone" : { "type" : "Home", "number" : "111-000-000" } }, "salary" : 85000, "doj" : ISO
skills" : [ "React", "MongoDB", "Javascript" ], "isActive" : true }
{ "_id" : ObjectId("629118c67017e180fa9ff11e"), "name" : { "firstName" : "Roma", "last
tName" : "Singla" }, "address" : { "phone" : { "type" : "Home", "number" : "9876543210
" }, "salary" : 80000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React",
"MongoDB", "Javascript" ], "isActive" : true }
{ "_id" : ObjectId("629f87ea863729e229a35888"), "name" : { "firstName" : "Rohit", "last
tName" : "Khurana" }, "address" : { "phone" : { "type" : "Home", "number" : "987654321
0" } }, "salary" : 30000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React
", "MongoDB", "Javascript" ], "isActive" : true }
>

```

## 2. \$set Stage

**\$set** stage is used to add new fields to the existing documents. It returns documents that contain all the existing fields and newly added fields. It is the same as that of the **\$addFields** stage. **If the name of the new field is the same as an existing field (including `_id`), \$set overwrites the existing value of that field with the new value.**

**Note:** \$set stage is new staging in version 4.2.

#### Syntax:

```

db.<collection_name>.aggregate(
  [
    {
      $set: { <newField>: <expression>, ... }
    }
  ]
);

```

#### Example:

Let's suppose we have to Add a new field `company_name` with the value `'GFG'` to every document in the `'employee_details'` collection using the MongoDB aggregation pipeline.

### G H Raison College of Engineering & Management

(Formerly Known as G H Raison Institute of Engineering & Technology, Nagpur)

An Autonomous Institute Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur

Accredited by NAAC with "A+" Grade

Shraddha Park, B-37-39/1, MIDC, Hingna-Wadi Link Road, Nagpur-440016 (INDIA)

#### Query:

```
db.employee_details.aggregate([
  {
    $set: {"company_name": "GFG"}
  }
])
```

#### Output:

```

TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  powershell  +  -  [ ]  [X]  ^  X

> use company;
switched to db company
> db;
company
> db.employee_details.aggregate([
...     {
...         $set: {"company_name": "GFG"}
...     }
... ])
{ "_id" : ObjectId("629101177017e180fa9ff11c"), "name" : { "firstName" : "Reema",
"lastName" : "Singla" }, "address" : { "phone" : { "type" : "Home", "number" : "11
1-000-000" } }, "salary" : 85000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills
" : [ "React", "MongoDB", "Javascript" ], "company_name" : "GFG" }
{ "_id" : ObjectId("6291185b7017e180fa9ff11d"), "name" : { "firstName" : "Seema",
"lastName" : "Garg" }, "address" : { "phone" : { "type" : "Home", "number" : "111-
000-000" } }, "salary" : 85000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills"
: [ "React", "MongoDB", "Javascript" ], "company_name" : "GFG" }
{ "_id" : ObjectId("629118c67017e180fa9ff11e"), "name" : { "firstName" : "Romal",
"lastName" : "Singla" }, "address" : { "phone" : { "type" : "Home", "number" : 987
6543210 } }, "salary" : 80000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" :
[ "React", "MongoDB", "Javascript" ], "company_name" : "GFG" }
{ "_id" : ObjectId("629f87ea863729e229a35888"), "name" : { "firstName" : "Rohit",
"lastName" : "Khurana" }, "address" : { "phone" : { "type" : "Home", "number" : "9
876543210" } }, "salary" : 30000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills
" : [ "React", "MongoDB", "Javascript" ], "company_name" : "GFG" }
> [ ]
```

### 3. Sunset Stage

**Sunset** stage is used to remove/exclude fields from the documents. If you want to remove the field or fields from the embedded document then use the dot notation. Sunset stage is new staging in version 4.2.

#### Syntax:

- To remove a single field, The Sunset stage takes a string that specifies the field to remove.

```
db.<collection_name>.aggregate(
[
  {
    $unset: "<field>"
  }
])
```

- To remove multiple fields, The Sunset takes an array of fields to remove.

```
db.<collection_name>.aggregate(
[
  {
    $unset: [ "<field1>", "<field2>", ... ]
  }
])
```

#### G H Raison College of Engineering & Management

(Formerly Known as G H Raison Institute of Engineering & Technology, Nagpur)

An Autonomous Institute Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur

Accredited by NAAC with "A+" Grade

Shraddha Park, B-37-39/1, MIDC, Hingna-Wadi Link Road, Nagpur-440016 (INDIA)

### Example:

Let's suppose we have to Remove the nested field `address.phone.type` from every document in the `employee\_details` collection using the MongoDB aggregation pipeline.

### Query:

```
db.employee_details.aggregate([
{
  $unset: "address.phone.type"
}
]);
```

### Output:

```

TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE
[> powershell + v [] [X] ^ X]

> use company
switched to db company
> db;
company
> db.employee_details.aggregate([
...   {
...     $unset: "address.phone.type"
...   }
... ]);
{ "_id" : ObjectId("629101177017e180fa9ff11c"), "name" : { "firstName" : "Reema",
"lastName" : "Singla" }, "address" : { "phone" : { "number" : "111-000-000" } }, "
salary" : 85000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React", "M
ongoDB", "Javascript" ] }
{ "_id" : ObjectId("6291185b7017e180fa9ff11d"), "name" : { "firstName" : "Seema",
"lastName" : "Garg" }, "address" : { "phone" : { "number" : "111-000-000" } }, "sa
lary" : 85000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React", "Mon
goDB", "Javascript" ] }
{ "_id" : ObjectId("629118c67017e180fa9ff11e"), "name" : { "firstName" : "Roma",
"lastName" : "Singla" }, "address" : { "phone" : { "number" : "9876543210" } }, "sal
ary" : 80000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React", "Mong
oDB", "Javascript" ] }
{ "_id" : ObjectId("629f87ea863729e229a35888"), "name" : { "firstName" : "Rohit",
"lastName" : "Khurana" }, "address" : { "phone" : { "number" : "9876543210" } }, "
salary" : 30000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React", "M
ongoDB", "Javascript" ] }
>

```

## 4. \$project Stage

**\$project** stage is used to include fields, exclude `_id` field, add new fields and replace the existing field value in the output document. The following is the table that describes the specification

**G H Raison College of Engineering & Management**

(Formerly Known as G H Raison Institute of Engineering & Technology, Nagpur)

An Autonomous Institute Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur

Accredited by NAAC with "A+" Grade

Shraddha Park, B-37-39/1, MIDC, Hingna-Wadi Link Road, Nagpur-440016 (INDIA)

## and its description.

| Specification        | Description                                                                                                                               |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| field : <1 or true>  | include the field in the output document                                                                                                  |
| _id : <0 or false>   | exclude the _id from the output document                                                                                                  |
| field : <expression> | add the new field or replace the existing field value                                                                                     |
| field : <0 or false> | exclude the field from the output document.<br>If you use the field to exclude instead of _id then you can't use any other specification. |

### Syntax:

```
db.<collection_name>.aggregate(
[
{
  $project: { <specification> }
}
])
```

### Example:

Let's suppose we have to Retrieve only the 'firstName' and 'lastName' fields from the 'name' object and the 'salary' field for every document in the 'employee\_details' collection, excluding the '\_id' field, using the MongoDB aggregation pipeline.

### Query:

```
db.employee_details.aggregate(
[
{
  $project: {"name.firstName":1, "name.lastName":1, "salary":1, "_id":0}
}
]);
```

### Output:

```

TERMINAL    PROBLEMS    OUTPUT    DEBUG CONSOLE
> use company;
switched to db company
> db;
company
> db.employee_details.aggregate(
...  [
...    {
...      $project: {"name.firstName":1, "name.lastName":1, "salary":1, "_id":0}
...    }
...  ]);
{ "name" : { "firstName" : "Reema", "lastName" : "Singla" }, "salary" : 85000 }
{ "name" : { "firstName" : "Seema", "lastName" : "Garg" }, "salary" : 85000 }
{ "name" : { "firstName" : "Romil", "lastName" : "Singla" }, "salary" : 80000 }
{ "name" : { "firstName" : "Rohit", "lastName" : "Khurana" }, "salary" : 30000 }
>
```

## G H Raison College of Engineering & Management

(Formerly Known as G H Raison Institute of Engineering & Technology, Nagpur)

An Autonomous Institute Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur

Accredited by NAAC with "A+" Grade

Shraddha Park, B-37-39/1, MIDC, Hingna-Wadi Link Road, Nagpur-440016 (INDIA)

## 5. \$count Stage

[\\$count](#) returns the count of the documents present in the selected collection.

**Note:** To get count of the documents in the collection we can also use `db.<collection_name>.count()`. It only returns the count we can't specify any output field.

### Syntax:

```
db.<collection_name>.aggregate(
  [
    {
      $count: <String>
    }
  ]
);
```

<string> is the name of the output field which has the count as its value.

### Example:

Let's suppose we have to Count the total number of documents in the 'employee\_details' collection and return the result as 'employees\_count' using the MongoDB aggregation pipeline.

### Query:

```
db.employee_details.aggregate(
  [
    { $count: "employees_count" }
  ]
);
```

### Output:

#### TERMINAL

#### PROBLEMS

#### OUTPUT

#### DEBUG CONSOLE

```
> use company;
switched to db company
> db;
company
> db.employee_details.aggregate(
...   [
...     { $count: "employees_count" }
...   ]
... );
{ "employees_count" : 4 }
>
```

## 6. \$unwind Stage

[\\$unwind](#) stage is used to deconstruct an array field from the input documents to output documents. If mentioned field path is not an array but is also not missing, null, or an empty array then \$unwind treats the non-array field as a single element array and the includeArrayIndex for that field is null.

**Note:** \$unwind stage does not include documents whose field is missing, null or empty array until you use `preserveNullAndEmptyArrays : true`

### Syntax:

```
db.<collection_name>.aggregate(
  [
    {
      $unwind: <String>
    }
  ]
);
```

## G H Raison College of Engineering & Management

(Formerly Known as G H Raison Institute of Engineering & Technology, Nagpur)

An Autonomous Institute Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur

Accredited by NAAC with "A+" Grade

Shraddha Park, B-37-39/1, MIDC, Hingna-Wadi Link Road, Nagpur-440016 (INDIA)

```

    {
      $unwind:
      {
        path: <field path>,
        includeArrayIndex: <string>,
        preserveNullAndEmptyArrays: <boolean>
      }
    }
  });

```

- Path specifies which field you want to unwind.
- includeArrayIndex includes array index only for array, not an application for fields that are not an array, null or empty.
- preserveNullAndEmptyArrays, If true then this option is used to include documents whose field is missing, null or empty array field. By default it is false.

#### Example:

Let's suppose we have to Skip the first 3 documents and then unwind the 'skills' array field in each remaining document in the 'employee\_details' collection using the MongoDB aggregation pipeline.

#### Query:

```

db.employee_details.aggregate(
[
  { $skip : 3 },
  { $unwind: "$skills" }
]
);

```

#### Output:

```

TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  powershell  + -  [ ]  [X]  [Y]  [Z]
> use company;
switched to db company
> db;
company
> db.employee_details.aggregate(
...   [
...     { $skip : 3 },
...     { $unwind: "$skills" }
...   ]
... );
{ "_id" : ObjectId("629f87ea863729e229a35888"), "name" : { "firstName" : "
Rohit", "lastName" : "Khurana" }, "address" : { "phone" : { "type" : "Home
", "number" : "9876543210" } }, "salary" : 30000, "doj" : ISODate("2022-05
-26T18:30:00Z"), "skills" : "React" }
{ "_id" : ObjectId("629f87ea863729e229a35888"), "name" : { "firstName" : "
Rohit", "lastName" : "Khurana" }, "address" : { "phone" : { "type" : "Home
", "number" : "9876543210" } }, "salary" : 30000, "doj" : ISODate("2022-05
-26T18:30:00Z"), "skills" : "MongoDB" }
{ "_id" : ObjectId("629f87ea863729e229a35888"), "name" : { "firstName" : "
Rohit", "lastName" : "Khurana" }, "address" : { "phone" : { "type" : "Home
", "number" : "9876543210" } }, "salary" : 30000, "doj" : ISODate("2022-05
-26T18:30:00Z"), "skills" : "Javascript" }
> █

```

The following is the output of the above operation. In this, we break down the document on the basis of an array field that is skills.

### G H Raison College of Engineering & Management

(Formerly Known as G H Raison Institute of Engineering & Technology, Nagpur)

An Autonomous Institute Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur

Accredited by NAAC with "A+" Grade

Shraddha Park, B-37-39/1, MIDC, Hingna-Wadi Link Road, Nagpur-440016 (INDIA)

## 7. \$limit Stage

\$limit stage is used to limit the number of documents passed to the next stage in the pipeline.

### Syntax:

```
db.<collection_name>.aggregate(  
  [  
    {  
      $limit: <positive 64-bit integer>  
    }  
  ]  
);
```

### Example:

Let's suppose we have to Limit the results to the first 2 documents and then count these documents, returning the result as 'employees\_count', using the MongoDB aggregation pipeline.

### Query:

```
db.employee_details.aggregate(  
  [  
    { $limit : 2 },  
    { $count : "employees_count" }  
  ]  
);
```

### Output:

TERMINAL

PROBLEMS

OUTPUT

DEBUG CONSOLE

```
> use company;  
switched to db company  
> db;  
company  
> db.employee_details.aggregate(  
...   [  
...       { $limit : 2 },  
...       { $count : "employees_count" }  
...   ] );  
{ "employees_count" : 2 }  
>
```

## 8. \$sort Stage

\$sort stage is used to sort the documents. If sorting is on multiple fields, the sort order is evaluated from left to right. For example, documents are first sorted by <field1> and then documents with the same <field1> values are further sorted by <field2>.

**Note:** You can sort a maximum of 32 keys.

### Syntax:

```
db.<collection_name>.aggregate(  
  [  
    {  
      $sort: { <field1> : <sort_order> }  
    }  
  ]  
);
```

```
{ $sort: { <field1>: <sort order>, <field2>: <sort order> ... } }
});
```

<field> represents the document field on the basis of which we sort the document either ascending or descending depends on the <sort-order>. <sort order> has the following values :

| Value | Order      |
|-------|------------|
| 1     | Ascending  |
| -1    | Descending |

### Example of \$sort in Ascending Order

Let's suppose we have to Sort the documents in the `'employee_details'` collection in ascending order based on the `'firstName'` field in the `'name'` object using the MongoDB aggregation pipeline.

**Query:**

```
db.employee_details.aggregate([
  {$sort: {"name.firstName": 1}}
]);
```

**Output:**

[TERMINAL](#)
[PROBLEMS](#)
[OUTPUT](#)
[DEBUG CONSOLE](#)

```
> use company;
switched to db company
> db;
company
> db.employee_details.aggregate(
...     [
...         {$sort : {"name.firstName" : 1}}
...     ]);
{ "_id" : ObjectId("6291185b7017e180fa9ff11d"), "name" : { "firstName" : "Aarti", "lastName" : "Garg" }, "address" : { "phone" : { "type" : "Home", "number" : "111-000-000" } }, "salary" : 85000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React", "MongoDB", "Javascript" ] }
{ "_id" : ObjectId("629f87ea863729e229a35888"), "name" : { "firstName" : "Rohit", "lastName" : "Khurana" }, "address" : { "phone" : { "type" : "Home", "number" : "9876543210" } }, "salary" : 30000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React", "MongoDB", "Javascript" ] }
{ "_id" : ObjectId("629118c67017e180fa9ff11e"), "name" : { "firstName" : "Romal", "lastName" : "Singla" }, "address" : { "phone" : { "type" : "Home", "number" : "9876543210" } }, "salary" : 80000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React", "MongoDB", "Javascript" ] }
{ "_id" : ObjectId("629101177017e180fa9ff11c"), "name" : { "firstName" : "Tina", "lastName" : "Singla" }, "address" : { "phone" : { "type" : "Home", "number" : "111-000-000" } }, "salary" : 85000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React", "MongoDB", "Javascript" ] }
>
```

### Example of \$sort in Descending Order

Let's suppose we have to Sort the documents in the `'employee_details'` collection in descending order based on the `'firstName'` field in the `'name'` object using the MongoDB aggregation pipeline.

**Query:**

```
db.employee_details.aggregate(  
  [  
    {$sort : {"name.firstName" : -1}}  
  ]  
);
```

### Output:

```

> use company;
switched to db company
> db;
company
> db.employee_details.aggregate(
...     [
...         {$sort : {"name.firstName" : -1}}
...     ]);
{ "_id" : ObjectId("629101177017e180fa9ff11c"), "name" : { "firstName" : "Tina", "lastName" : "Singla" }, "address" : { "phone" : { "type" : "Home", "number" : "111-000-000" } }, "salary" : 85000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React", "MongoDB", "Javascript" ] }
{ "_id" : ObjectId("629118c67017e180fa9ff11e"), "name" : { "firstName" : "Roma", "lastName" : "Singla" }, "address" : { "phone" : { "type" : "Home", "number" : "9876543210" } }, "salary" : 80000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React", "MongoDB", "Javascript" ] }
{ "_id" : ObjectId("629f87ea863729e229a35888"), "name" : { "firstName" : "Rohit", "lastName" : "Khurana" }, "address" : { "phone" : { "type" : "Home", "number" : "9876543210" } }, "salary" : 30000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React", "MongoDB", "Javascript" ] }
{ "_id" : ObjectId("6291185b7017e180fa9ff11d"), "name" : { "firstName" : "Aarti", "lastName" : "Garg" }, "address" : { "phone" : { "type" : "Home", "number" : "111-000-000" } }, "salary" : 85000, "doj" : ISODate("2022-05-26T18:30:00Z"), "skills" : [ "React", "MongoDB", "Javascript" ] }
>

```

## 9. \$sortByCount Stage

**\$sortByCount** stage is used to group the documents on the basis of field path (document field) and then count the number of documents in each different group and the documents are sorted by count in descending order. The \$sortByCount stage returns the documents. Each document has two fields '\_id' field containing the different grouping values and the 'count' field containing the number of documents belonging to that group.

**Note:** The `$sortByCount` stage is equivalent to the following: `$group + $sort` sequence: { `$group`: { `_id`: <expression>, `count`: { `$sum`: 1 } } }, { `$sort`: { `count`: -1 } }

**Syntax:**

```
db.<collection_name>.aggregate([
  { $sortByCount : "<field path>" }
]);
```

**Example:**

Let's suppose we have to Create a MongoDB aggregation pipeline to list and count the occurrences of each skill across all employees in the `employee\_details` collection after unwinding the `skills` array field.

**Query:**

```
db.employee_details.aggregate([
  { $unwind : "$skills" },
  { $sortByCount : "$skills" }
]);
```

### Output:

**TERMINAL****PROBLEMS****OUTPUT****DEBUG CONSOLE**

```
> use company;
switched to db company
> db;
company
> db.employee_details.aggregate(
...   [
...     {$unwind : "$skills"},
...     {$sortByCount : "$skills"}
...   ]);
{ "_id" : "Javascript", "count" : 4 }
{ "_id" : "MongoDB", "count" : 4 }
{ "_id" : "React", "count" : 2 }
{ "_id" : "Java", "count" : 2 }
>
```

## 10. \$skip Stage

**\$skip** stage is used to skip the number of documents and passes the remaining documents to the next stage in the pipeline. It returns the remaining documents.

**Syntax:**

```
db.<collection_name>.aggregate(
[
  { $skip: <positive 64-bit integer> }
]);
```

**Example:**

Let's suppose we have to skip the first 3 documents in the 'employee\_details' collection using the MongoDB aggregation pipeline.

**Query:**

```
db.employee_details.aggregate(
[
  { $skip : 3 }
]);
```

**Output:**

```

TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  powershell  +  -  [  ]  X
> use company;
switched to db company
> db;
company
> db.employee_details.aggregate(
...   [
...     {$skip : 3}
...   ]);
{ "_id" : ObjectId("629f87ea863729e229a35888"), "name" : { "firstName" :
"Rohit", "lastName" : "Khurana" }, "address" : { "phone" : { "type" : "Ho
me", "number" : "9876543210" } }, "salary" : 30000, "doj" : ISODate("2022
-05-26T18:30:00Z"), "skills" : [ "React", "MongoDB", "Javascript" ] }
> █

```

### How Fast is MongoDB Aggregation?

- The speed of MongoDB aggregation depends on various factors such as the complexity of the aggregation pipeline, the size of the data set, the hardware specifications of the **MongoDB server** and the efficiency of the indexes.
- In general, MongoDB's aggregation framework is designed to efficiently process large volumes of data and complex aggregation operations.
- When used correctly it can provide fast and scalable aggregation capabilities.
- So with any [database](#) operation, the performance can vary based on the specific use case and configuration.
- It is important to optimize our aggregation queries and use indexes where appropriate and ensure that our MongoDB server is properly configured for optimal performance

## MongoDB \$group aggregation operators

The \$group stage supports certain expressions (operators) allowing users to perform arithmetic, array, boolean and other operations as part of the aggregation pipeline.

| Operator | Meaning                                                              |
|----------|----------------------------------------------------------------------|
| \$count  | Calculates the quantity of documents in the given group.             |
| \$max    | Displays the maximum value of a document' s field in the collection. |
| \$min    | Displays the minimum value of a document' s field in the collection. |

### G H Raison College of Engineering & Management

(Formerly Known as G H Raison Institute of Engineering & Technology, Nagpur)

An Autonomous Institute Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur

Accredited by NAAC with "A+" Grade

Shraddha Park, B-37-39/1, MIDC, Hingna-Wadi Link Road, Nagpur-440016 (INDIA)

|        |                                                                     |
|--------|---------------------------------------------------------------------|
| \$avg  | Displays the average value of a document's field in the collection. |
| \$sum  | Sums up the specified values of all documents in the collection.    |
| \$push | Adds extra values into the array of the resulting document.         |

## What is the Aggregation Framework?

**The Aggregation Framework in MongoDB allows you to process and transform data in collections using an aggregation pipeline.** This pipeline consists of one or more stages, where each stage takes the output of the previous stage as its input and performs a specific operation.

## How it Works (The Pipeline Concept)

Think of it like an assembly line where a raw product goes through several stations, with each station performing a specific task to transform the product. In the aggregation framework:

1. **Input:** The pipeline starts with documents from a MongoDB collection.
2. **Stages:** Each stage performs a specific operation, such as filtering, grouping, or transforming documents.
3. **Output:** The output of one stage becomes the input for the next stage, moving the data through the pipeline until the final aggregated result is produced.

## Common Aggregation Stages

There are many aggregation stages available, but some of the most frequently used ones are:

- **\$match:** Filters documents to pass only those that meet specified criteria to the next stage. Placing this at the beginning of the pipeline can improve performance by reducing the number of documents that need further processing.
- **\$group:** Groups input documents by a specified key (like a field) and applies accumulator expressions to summarize the grouped data.
- **\$sort:** Reorders the documents based on a specified field.

**\$project:** Reshapes

each document by including or excluding specific fields, or adding new fields derived from existing data.

- **\$limit**: Limits the number of documents passed to the next stage or the final output.
- **\$skip**: Skips a specified number of documents.
- **\$unwind**: Deconstructs an array field from an input document to output a new document for each element in the array.
- **\$lookup**: Performs a left-outer join to include data from another collection, allowing you to incorporate related data into the pipeline.
- **\$out**: Writes the results of the aggregation pipeline to a new collection.

## MongoDB Map-Reduce

**MongoDB Map-Reduce is a data processing model that facilitates operations on large data sets to produce aggregated results.** It uses the `mapReduce()` function comprising map and reduce functions to handle complex data transformations.

### What MongoDB Map-Reduce

**MongoDB Map-Reduce** is a data processing programming model that helps to perform operations on large data sets and produce **aggregated** results. MongoDB provides the **mapReduce()** function to perform the map-reduce operations. This function has two main functions, i.e., **map** function and **reduce** function.

The map function is used to group all the data based on the **key-value** and the **reduce function is used to perform operations on the mapped data**. So, the data is **independently** mapped and reduced in different spaces and then combined in the function and the result will be saved to the specified new collection.

**Map-Reduce is a powerful feature in MongoDB for aggregating data.**

### Syntax

```
db.collectionName.mapReduce(... map(),...reduce(),...query{ },...output{ });
```

### Key Terms:

- **map() function**: It uses the **emit()** function in which it takes two parameters **key** and **value key**. Here the key is on which we make groups like Group By in MySQL.
  - **reduce() function**: It is the step in which we perform our aggregate functions like `avg()`, and `sum()`.
  - **query**: we will pass the query to filter the resultset.
  - **output**: we will specify the collection name where the result will be stored
- . In this example, we have five records from which we need to take out the maximum marks of each section and the **keys are id, sec, marks**.

```
{"id":1, "sec":A, "marks":80}  
{"id":2, "sec":A, "marks":90}  
{"id":1, "sec":B, "marks":99}  
{"id":1, "sec":B, "marks":95}  
{"id":1, "sec":C, "marks":90}
```

Here we need to **find the maximum marks in each section**. So, our key by which **we will group documents is the sec key and the value will be marks**. Inside the map function, we **use emit(this.sec, this.marks) function, and we will return the sec and marks of each record(document) from the emit function**. This is similar to group By MySQL.

```
var map = function(){emit(this.sec, this.marks)};
```

After iterating over each document Emit function will give back the data like this:

```
{"A":[80, 90]}, {"B":[99, 90]}, {"C":[90]} }
```

The data given by emit function is grouped by sec key, Now this data will be input to our reduce function. **Reduce function is where actual aggregation of data takes place**. In our example we will pick the Max of each section like for  
sec A:[80, 90] = 90 (Max) B:[99, 90] = 99 (max) , C:[90] = 90(max).

Here in reduce() function, we have reduced the records now we will output them into a new collection. {out : "collectionName"}

```
db.collectionName.mapReduce(map,reduce,{out : "collectionName"});
```

we can use the query [db.collectionName.find\(\)](#) we get:

```
{"id":"A", value:90}  
{"id":"B", value:99}  
{"id":"C", value:90}
```

### When to use Map-Reduce in MongoDB?

- **Large Datasets:** The aggregation query is slow due to the volume of data. Map-Reduce can process large datasets more efficiently.
- **Complex Aggregations:** If your aggregation operations are complex or not easily achievable using MongoDB's aggregation pipeline.
- **Custom Data Transformations:** We need the flexibility of custom JavaScript functions to process and aggregate the data.
- **Performance Optimization:** Map-Reduce can be faster for some types of data aggregation than the aggregation pipeline, especially for operations like summing, averaging, or finding maximum values over large datasets.

### When Not to Use MongoDB Map-Reduce

While MongoDB Map-Reduce is powerful, it is not always the best option. Avoid using it when:

- **Simple Aggregation Operations:** MongoDB's aggregation pipeline is generally more efficient for simple aggregation tasks like sum, count, and average.
- **Performance Concerns:** Map-Reduce involves more overhead and can be slower than the aggregation pipeline for simpler queries.

In MongoDB, the `$lookup` operation is used to perform a left outer join between two collections. It allows you to combine documents from one collection with related documents from another collection based on a specified field.

Syntax

```
{
  $lookup: {
    from: "<other_collection>", // Collection to join with
    localField: "<field_in_this_collection>", // Field from input
documents
    foreignField: "<field_in_other_collection>", // Field from the other
collection
    as: "<output_array_field>" // Name of the new array field to store
matching documents
  }
}
```

Example

Suppose we have two collections:

**1. orders collection:**

```
{ "_id": 1, "customerId": 101, "total": 250 }  
{ "_id": 2, "customerId": 102, "total": 150 }
```

**2. customers collection:**

```
{ "_id": 101, "name": "Alice" }  
{ "_id": 102, "name": "Bob" }
```

Aggregation query using \$lookup:

```
db.orders.aggregate([  
  {  
    $lookup: {  
      from: "customers",  
      localField: "customerId",  
      foreignField: "_id",  
      as: "customerInfo"  
    }  
  }  
)
```

Result:

```
{  
  "_id": 1,  
  "customerId": 101,  
  "total": 250,  
  "customerInfo": [{ "_id": 101, "name": "Alice" }]  
}  
{  
  "_id": 2,  
  "customerId": 102,  
  "total": 150,  
  "customerInfo": [{ "_id": 102, "name": "Bob" }]  
}
```

Key points:

\$lookup always produces an array in the as field, even if there' s only one matching document.

If no match is found, the array will be empty.

Can be combined with \$unwind if you want one-to-one flattening of joined data.